WARWICK

Advanced Computer Graphics

Light Transport and Path Tracing

Dr. Tom Bashford-Rogers

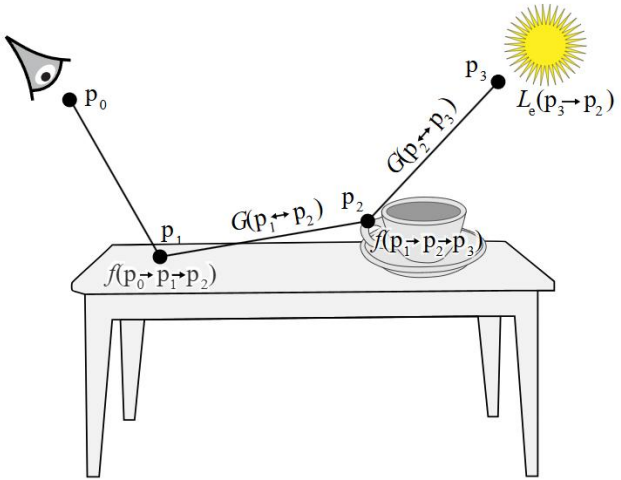
Overview

- Light Transport
- Direct Lighting
- Indirect Lighting

Light Transport

- Motivation

Light Transport



Images from https://pbr-book.org/

Light Transport

- Phenomena

Light Transport

- Common terms
  - Point in the scene (path vertex) x
  - Normal at a point n
  - Direction from x_1 to x_2: $x_1 \rightarrow x_2$
  - Incoming direction to a point ω_i
  - Outgoing direction from a point ω_o

Light Transport

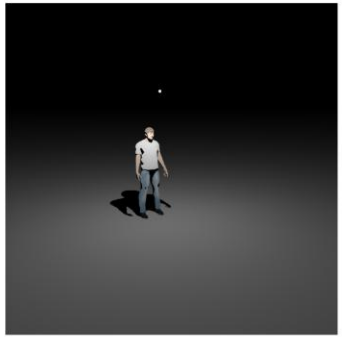
- Common terms
  - Hemisphere Ω
  - Sphere $\mathcal{S}^2$
  - Area A
  - Scene surfaces $\mathcal{M}$

Light Transport

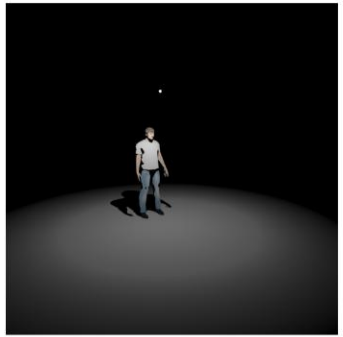
- Light
  - Radiance
  - Emitted from light sources $L_e(\mathbf{x}, \omega_o)$ or $L_e(x_0 \rightarrow x_1)$
  - Incoming Light $L_i(x, \omega_i)$ or $L_i(x_n \rightarrow x_{n+1})$
  - Outgoing Light $L_o(x, \omega_o)$ or $L_o(x_{n+1} \rightarrow x_n)$

Light Transport

- Light
  - Emitted from a point
  - Several types
    - Point / Spot / Directional
    - Area
    - Environment
    - More



Point Light



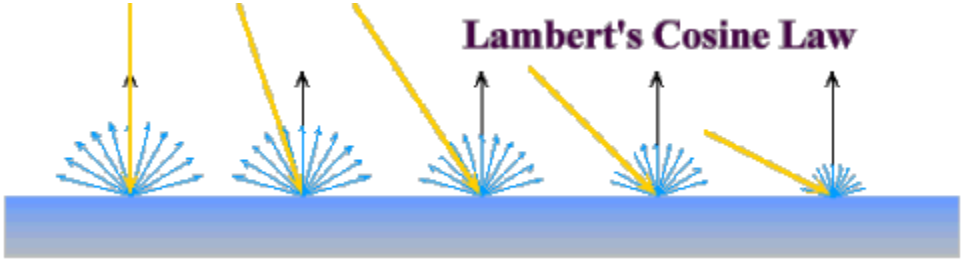
Spot Light

Area Light

Sky Light

Light Transport

- Cosine Term $(\omega_i \cdot n)$ or $\cos(\theta)$
- Models Lamberts Law



Light Transport

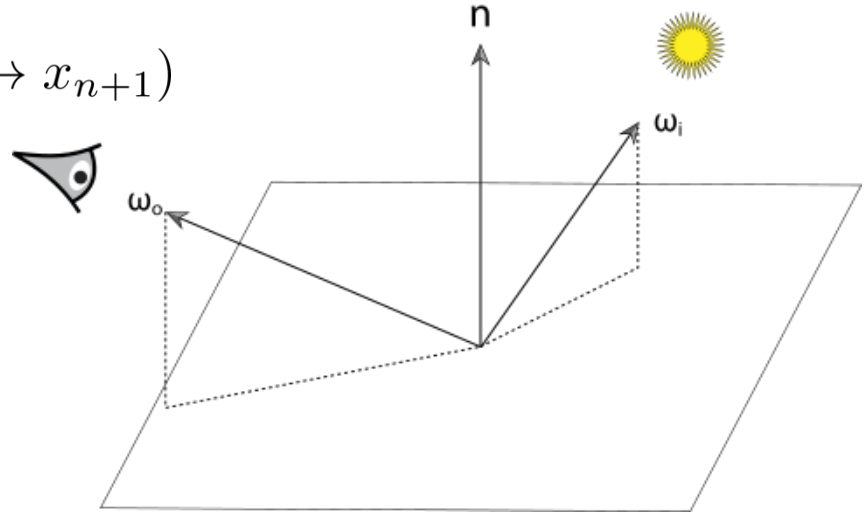
- Cosine Term
  - Incoming radiance (light) scaled by

$$L_e(x, \omega_i)(\omega_i \cdot n) = L_e(x, \omega_i)\cos(\theta)$$

  - Note this is different to cosine in radiance definition
    - Ensures radiance is measured per unit projected area
    - Here describes how brightness changes with angle

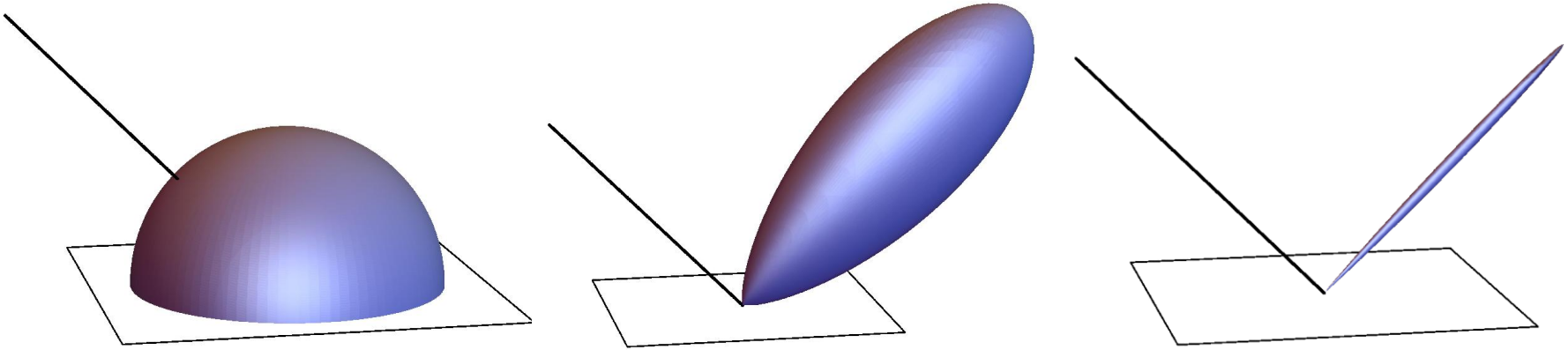
Light Transport

- Materials: BRDF
  - Bidirectional Reflectance Distribution Function
  - $f_r(\omega_i, \omega_o)$ or $f_r(x_{n-1} \rightarrow x_n \rightarrow x_{n+1})$



Light Transport

- Materials: BRDF



Light Transport

- Materials: BRDF

Light Transport

- Materials: BRDF
  - Consider ω_i as a differential cone of directions
  - Write differential irradiance as

$$dE(x, \omega_i) = L_i(x, \omega_i) \cos(\theta_i) d\omega_i$$

  - Know reflected radiance is proportional to this

$$dL_o(x, \omega_o) \propto dE(x, \omega_i)$$

Light Transport

- Materials: BRDF
  - BRDF is the proportionality factor

$$f_r(x, \omega_i, \omega_o) = \frac{dL_o(x, \omega_o)}{dE(x, \omega_i)}$$

  - Ratio of the differential reflected radiance in the direction ω_o to the differential irradiance arriving from the direction ω_i

$$f_r(x, \omega_i, \omega_o) = \frac{dL_o(x, \omega_o)}{L_i(x, \omega_i)\cos(\theta_i)d\omega_i}$$

Light Transport

- Materials: BRDF Properties
  - Units sr^{-1}
  - Helmholtz Reciprocity

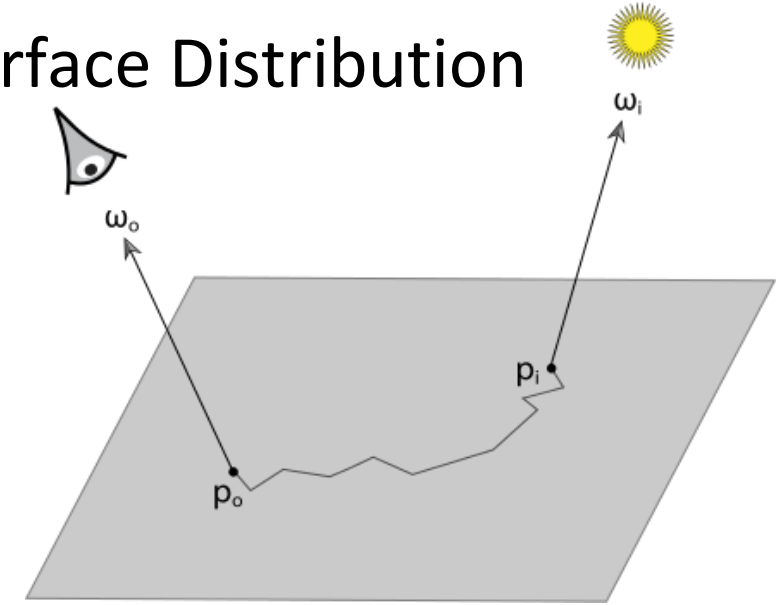
$$f_r(x, \omega_i, \omega_o) = f_r(x, \omega_o, \omega_i)$$

  - Energy Conservation

$$\int_\Omega f_r(x, \omega_i, \omega_o) \cos\theta_i \, d\omega_i \leq 1$$

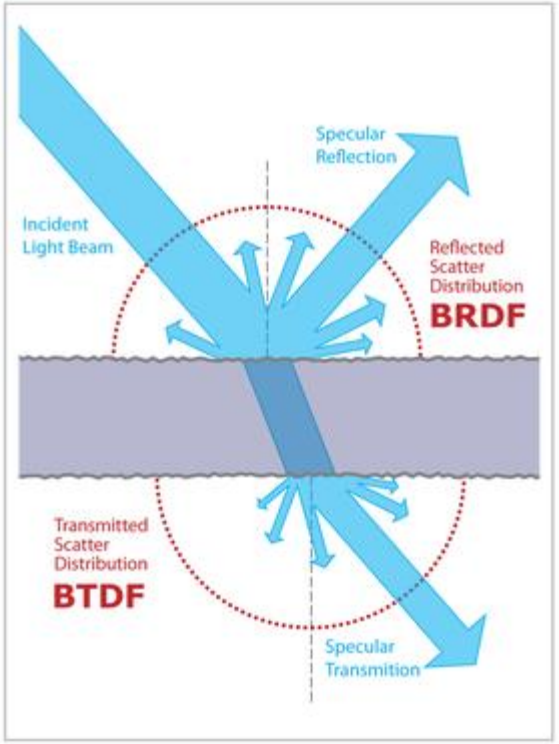
Light Transport

- Materials: BSSDF
  - Bidirectional Scattering Surface Distribution Function



Light Transport

- Materials: BSDF
  - Combines BRDF and BTDF
    - Bidirectional Transmission Distribution Function



Light Transport

- Rendering Equation

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_\Omega f_r(x, \omega_i, \omega_o) L_i(x, \omega_i)(\omega_i \cdot n) d\omega_i$$

Outgoing Radiance

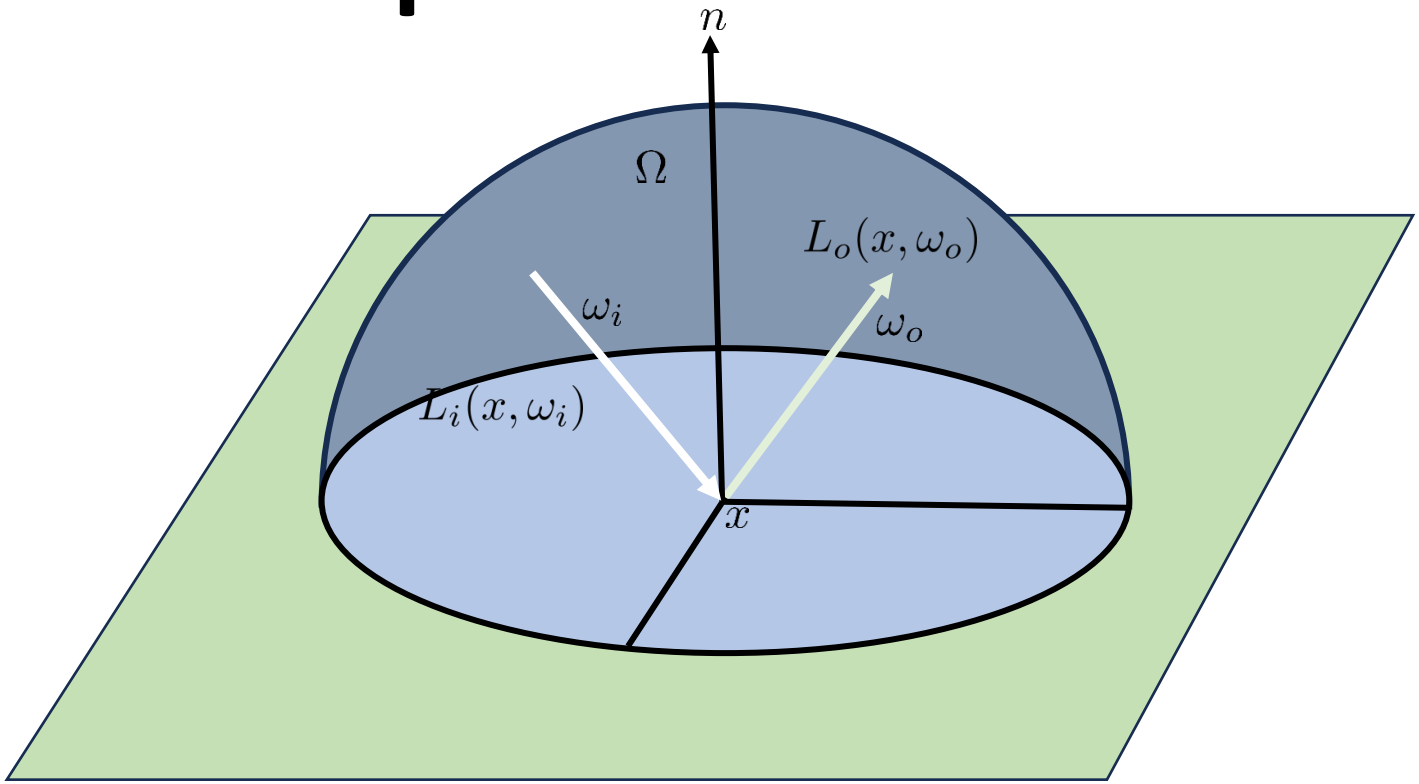
Emitted Radiance

BRDF

Incoming Radiance

Cosine Term

Light Transport



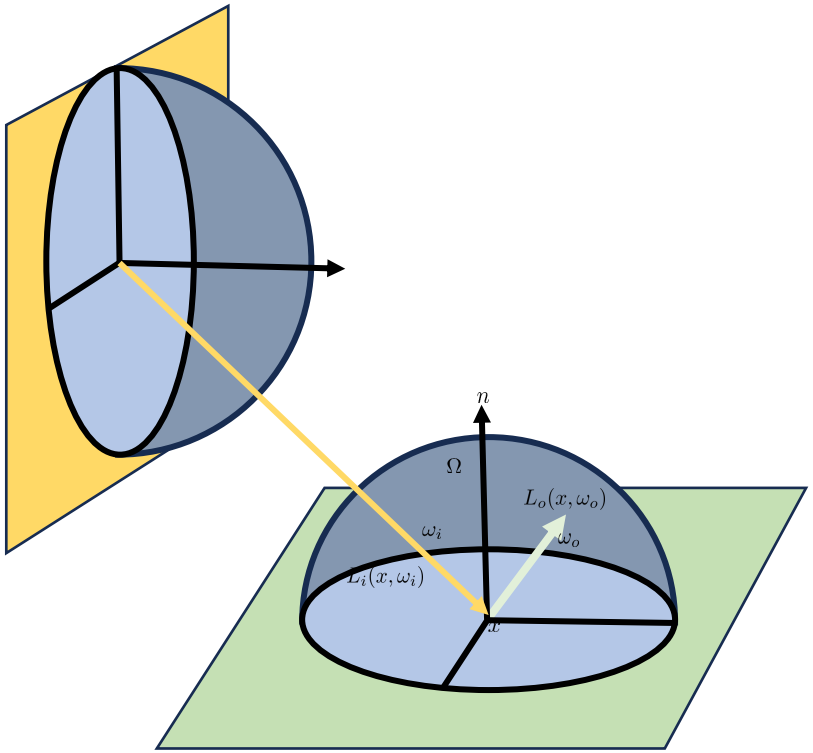
Light Transport

- Rendering Equation
  - Origin of $L_i(x, \omega_i)$
    - Next surface visible along the ray specified by x, ω_i
    - Define this to be $h(x, \omega_i)$
    - So $L_i(x, \omega_i) = L_o(h(x, \omega_i), -\omega_i)$
    - Therefore

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_\Omega f_r(x, \omega_i, \omega_o) L_o(h(x, \omega_i), -\omega_i)(\omega_i \cdot n) d\omega_i$$

Light Transport

- Rendering Equation
  - Recursive definition



$$L(x,\omega_o) = L_e(x,\omega_o) + \int_\Omega f_r(x,\omega_i,\omega_o) L(h(x,\omega_i),-\omega_i)(\omega_i \cdot n) d\omega_i$$

Light Transport

- Rendering Equation
  - Currently integrating over the hemisphere of incoming directions (solid angle)
  - But what if we want to integrate over scene surfaces?
    - We will see why later

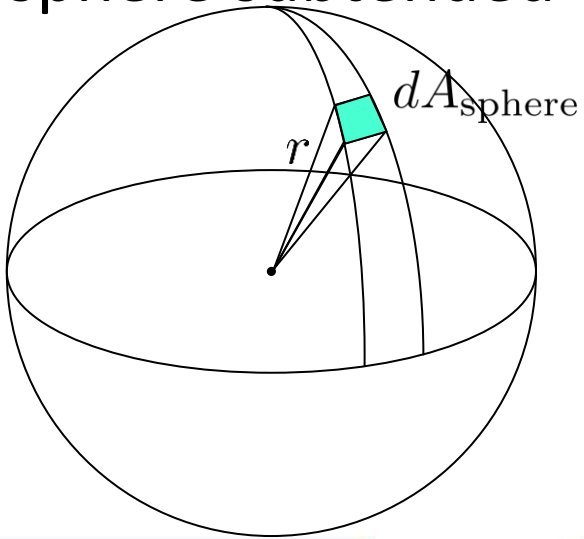
Light Transport

- Rendering Equation
  - Need to change integration domain to scene surfaces
  - Ω to $\mathcal{M}$
  - Need to incorporate change of variables

Light Transport

- Change of variables
  - Solid angle: area on the unit sphere subtended by a surface:

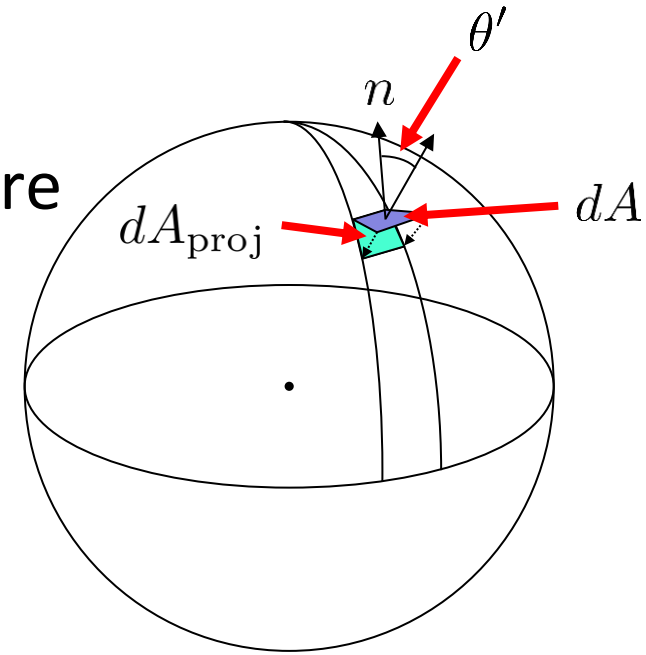
$$dA_{\text{sphere}} = r^2 \, d\Omega$$



Light Transport

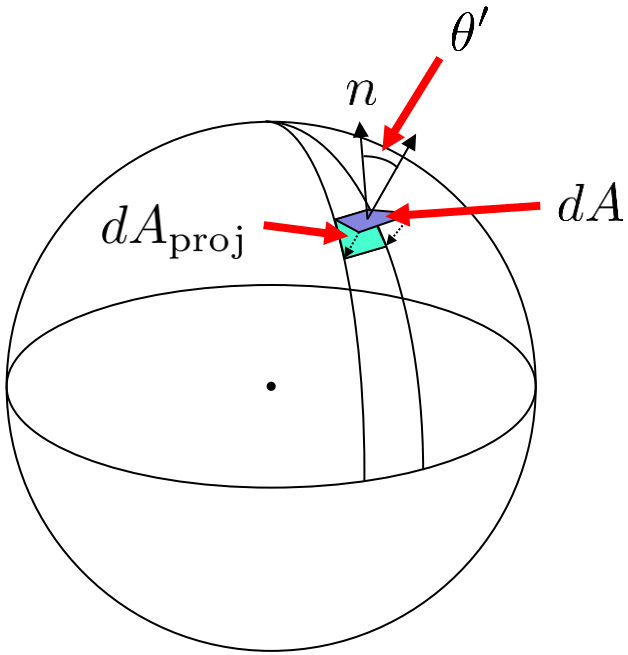
- Change of variables
  - Surface projected to sphere

$$dA_{\text{proj}} = dA \cos\theta'$$



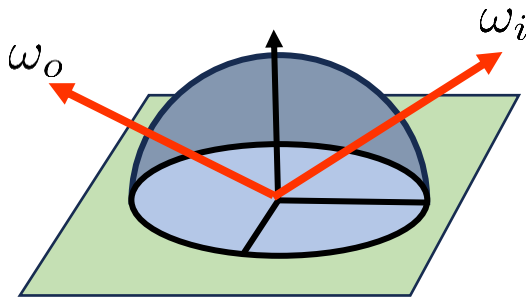
Light Transport

- Change of variables
  - Set equal $dA_{\text{sphere}} = dA_{\text{proj}}$
  - So $dA \cos\theta' = r^2 \, d\Omega.$
  - Rearrange $d\Omega = \dfrac{\cos\theta'}{r^2} dA$



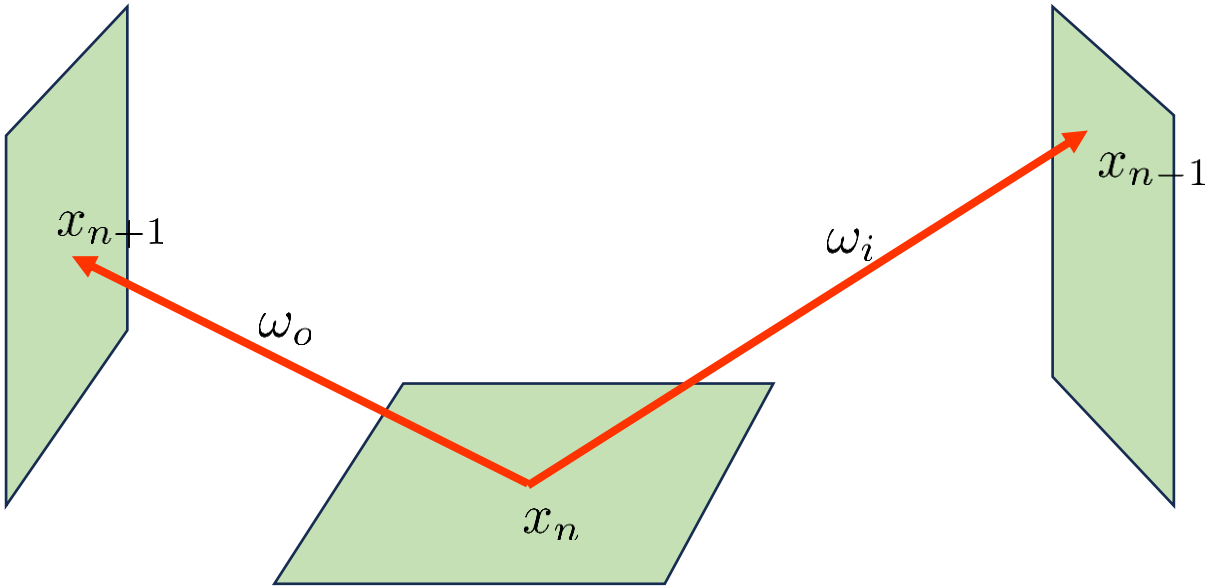
Light Transport

- Integrating over points on scene surfaces
  - Given the current point
  - So $\omega_i = x_{n-1} \rightarrow x_n$
  - And $\omega_o = x_n \rightarrow x_{n+1}$



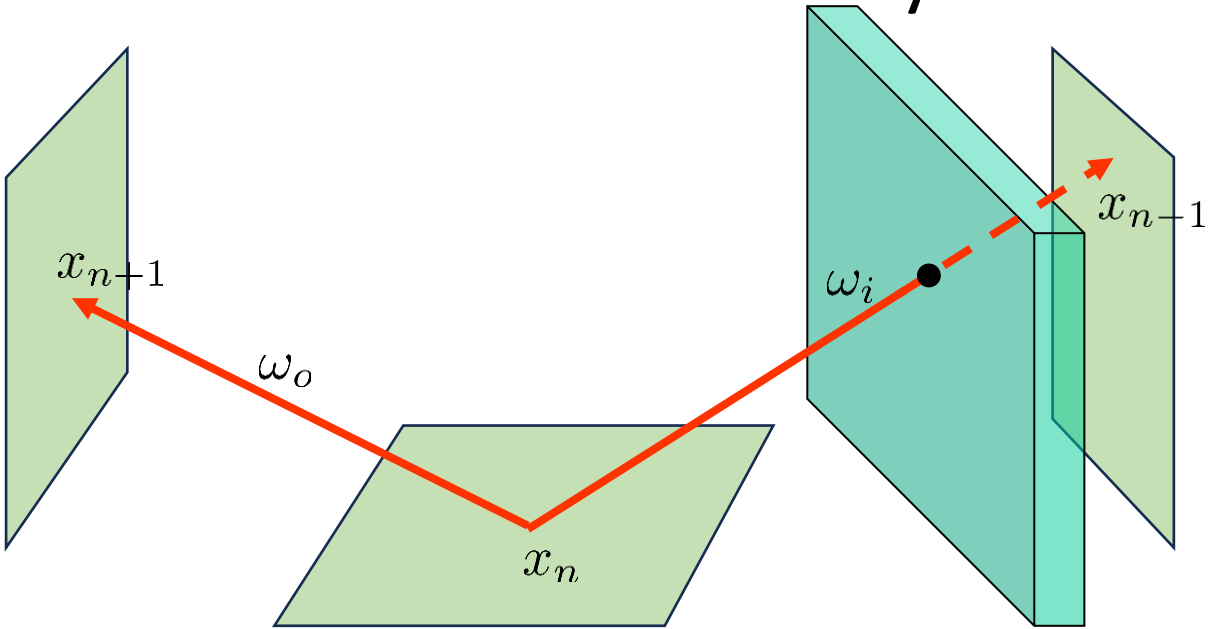
Light Transport

- Integrating over points on scene surfaces



Light Transport

- But we need to include visibility



Light Transport

- Visibility
  - Binary Function $V(x_i \leftrightarrow x_{i+1}) = \begin{cases} 1 & \text{if visible} \\ 0 & \text{otherwise} \end{cases}$
  - Computed with ray tracing
    - Can be faster than standard ray tracing, return hit as soon as any primitive is hit, do not need to find the nearest

Light Transport

- Combine common terms into the "Geometry Term"
  - Cosine Term $\cos(\theta)$
  - Change of variables $\frac{\cos(\theta')}{r^2}$
  - Visibility $V(x_i \leftrightarrow x_{i+1})$

Light Transport

- Geometry Term:

$$G(x_i \leftrightarrow x_{i+1}) = \frac{\cos(\theta)\cos(\theta')}{r^2} V(x_i \leftrightarrow x_{i+1})$$

- Or

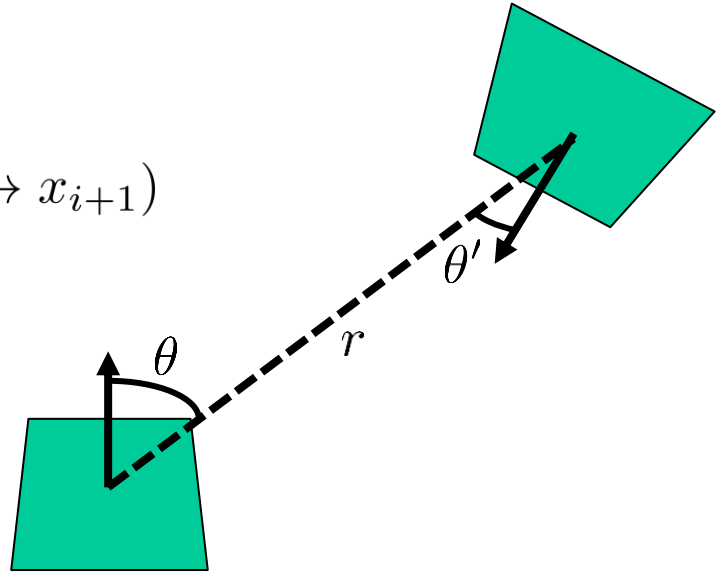
$$G(x_i \leftrightarrow x_{i+1}) = \frac{(\omega_i \cdot n)(-\omega_i \cdot n')}{||x_i - x_{i+1}||^2} V(x_i \leftrightarrow x_{i+1})$$

  - Where n is the normal at x_i
  - And n' is the normal at x_{i+1}

Light Transport

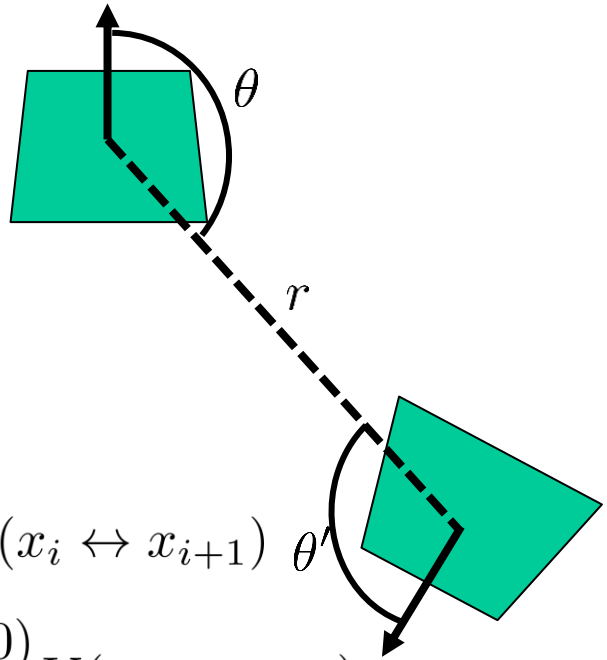
- Geometry Term

$$G(x_i \leftrightarrow x_{i+1}) = \frac{\cos(\theta)\cos(\theta')}{r^2} V(x_i \leftrightarrow x_{i+1})$$



Light Transport

- Geometry Term
  - How to handle surfaces not facing each other
  - Will be further generalized



$$G(x_i \leftrightarrow x_{i+1}) = \frac{\max(\cos(\theta), 0)\max(\cos(\theta'), 0)}{r^2} V(x_i \leftrightarrow x_{i+1})$$

$$G(x_i \leftrightarrow x_{i+1}) = \frac{\max((\omega_i \cdot n), 0)\max(-(\omega_i \cdot n'), 0)}{||x_i - x_{i+1}||^2} V(x_i \leftrightarrow x_{i+1})$$

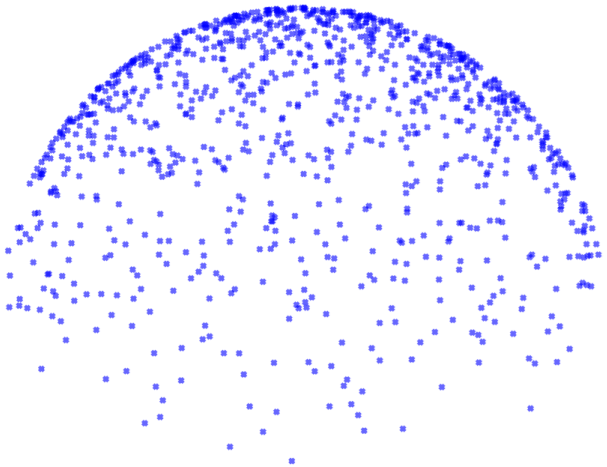
Light Transport

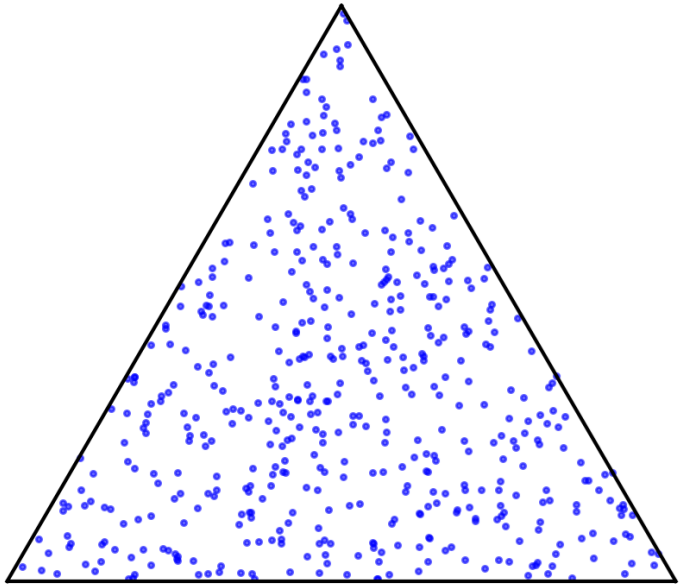
- Rendering Equation
  - Area form

$$L(x_n \to x_{n+1}) = L_e(x_n \to x_{n+1}) + \int_{\mathcal{M}} f_r(x_{n-1} \to x_n \to x_{n+1}) L(x_{n-1} \to x_n) G(x_{n-1} \leftrightarrow x_n) dA$$

Light Transport

- How to solve?
- Monte Carlo!





Light Transport

- Monte Carlo Estimates
  - Solid angle

$$L_o(x, \omega_o) \approx L_e(x, \omega_o) + \frac{1}{N} \sum_{j=1}^{N} \frac{f_r(x, \omega_i(j), \omega_o) L_i(x, \omega_i(j)) (\omega_i(j) \cdot n)}{p_\Omega(\omega_i(j))}$$

  - PDF can be any density over the hemisphere

Light Transport

- Monte Carlo Estimates
  - Area estimate

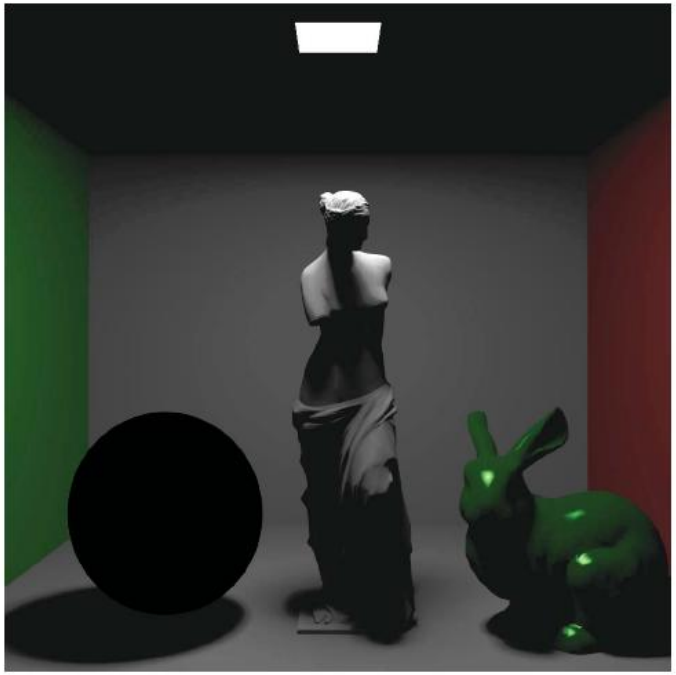
$$L(x_n \to x_{n+1}) = L_e(x_n \to x_{n+1}) +$$
$$\frac{1}{N}\sum_{j=1}^{N} \frac{f_r(x_{n-1}(j) \to x_n \to x_{n+1}) L(x_{n-1}(j) \to x_n) G(x_{n-1}(j) \leftrightarrow x_n)}{p_A(x_{n-1}(j))}$$

  - PDF for generating a point on a surface in the scene

Light Transport

- Monte Carlo Estimates
  - Can combine both
  - Use solid angle for some lighting effects
    - Indirect lighting
  - Use area sampling for others
    - Direct lighting

Light Transport

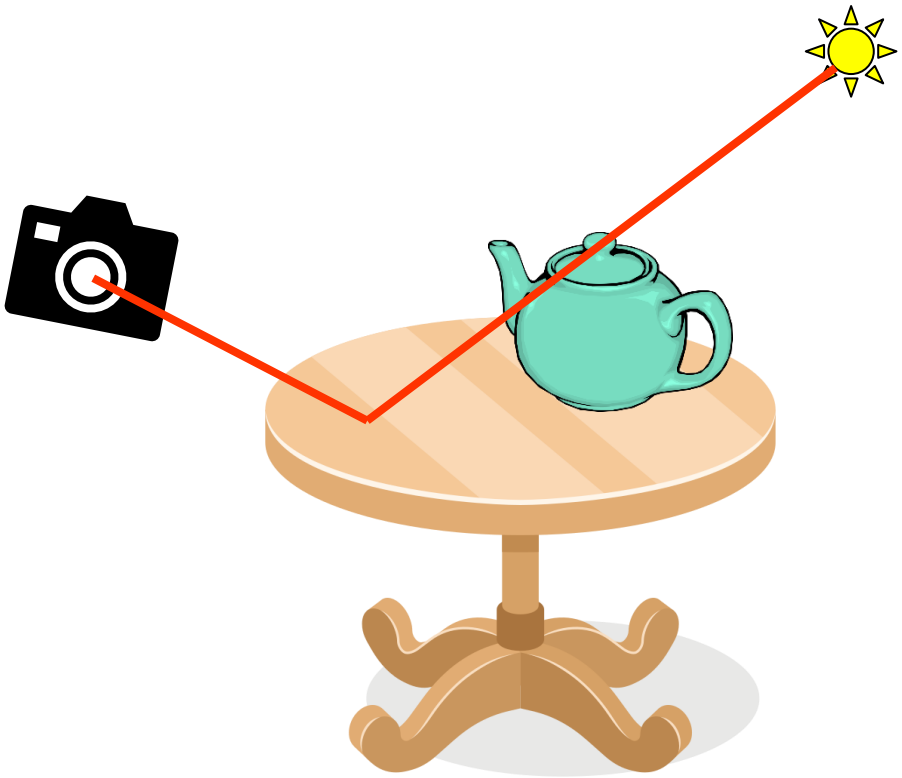


L_d

L_{ind}

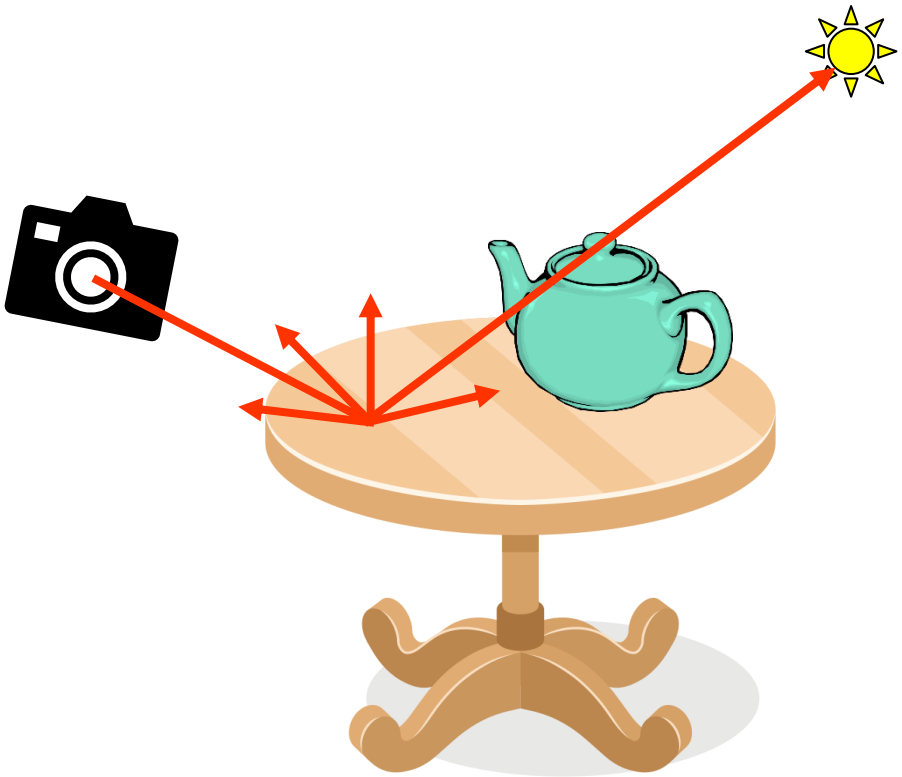
Light Transport

- Direct Lighting
  - Computes light interacting once with the scene between the light and camera



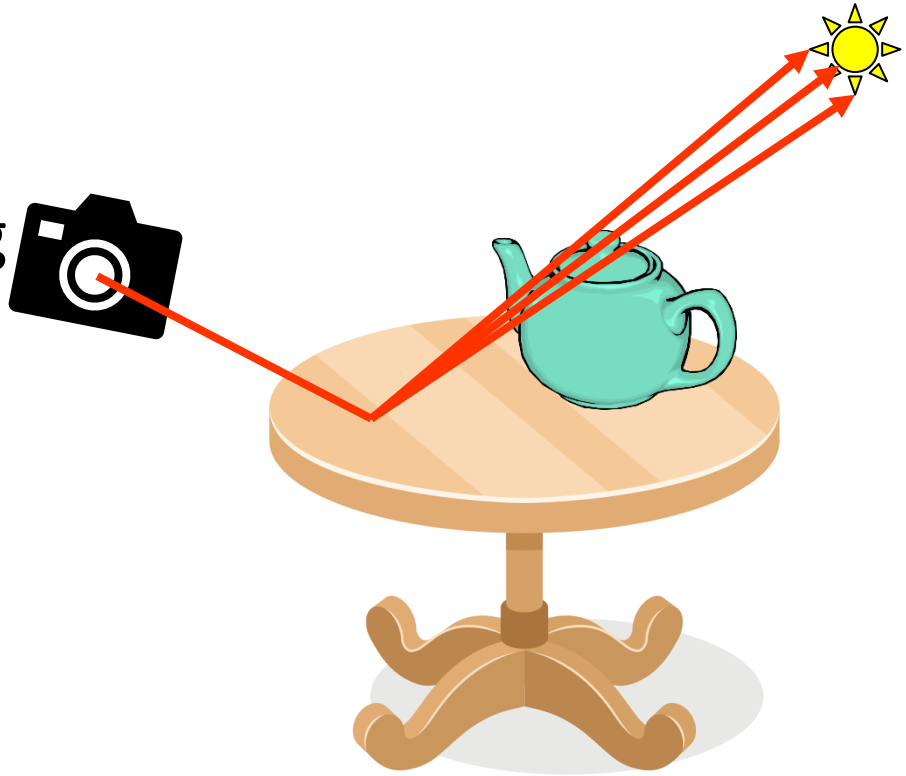
Light Transport

- Direct Lighting
  - Solid Angle Sampling
  - Very low chance of hitting light



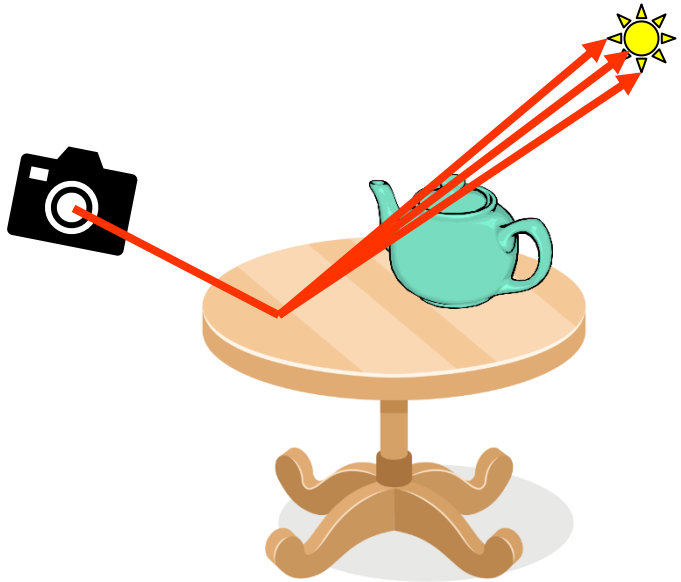
Light Transport

- Direct Lighting
  - Area Angle Sampling
  - Very high chance of hitting light
  - Known as Next Event Estimation (NEE)



Light Transport

- Direct Lighting
  - But has singularity from geometry term
  - Good for distant lights
  - Increased noise for very close lights



Light Transport

- Monte Carlo Estimates
  - Direct lighting to a point

$$L_d(x_1 \to x_{n+1}) =$$

$$\frac{1}{N}\sum_{j=1}^{N}\frac{f_r(x_0(j) \to x_1 \to x_2)L(x_0(j) \to x_1)G(x_0(j) \leftrightarrow x_1)}{p_A(x_0(j))}$$

Light Transport

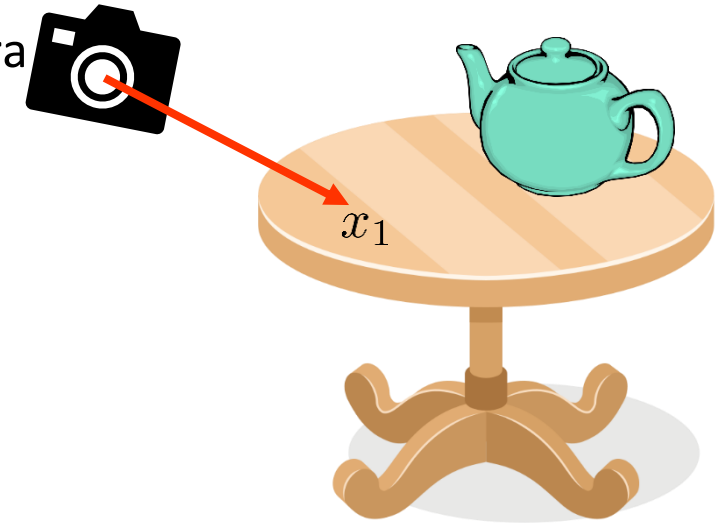
- How to implement this
  - Given a point in the scene x_1
  - Sample a light source l_i
  - Sample a point on the light source $x_0(j)$ / direction to the light source $\omega_i(j)$
  - Evaluate integrand

$$f_r(x_0(j) \to x_1 \to x_2) L(x_0(j) \to x_1) G(x_0(j) \leftrightarrow x_1)$$

  - Divide by PDF $p_A(x_{n-1}(j))$ or $p_\Omega(\omega_i(j))$

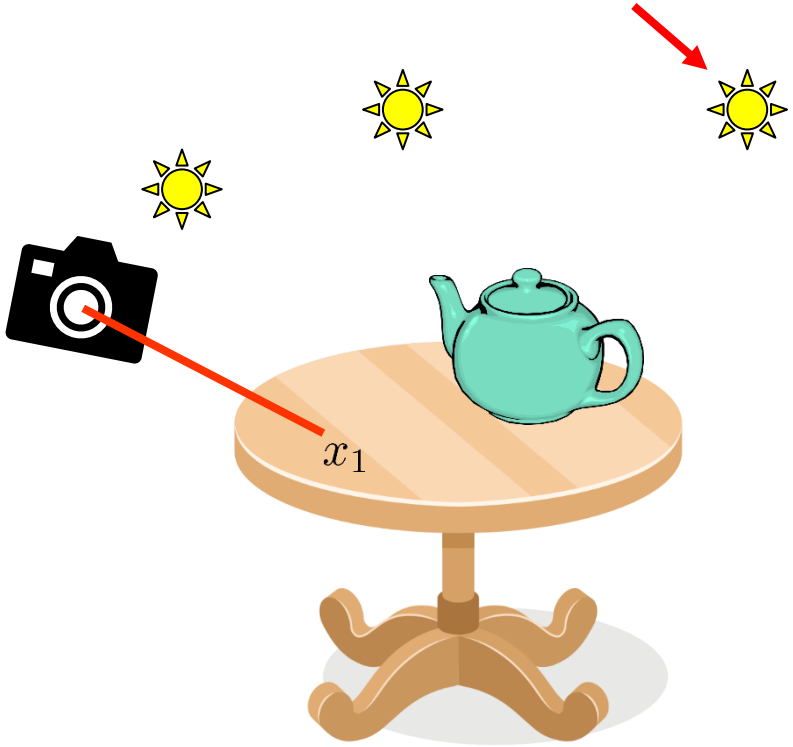
Light Transport

- How to implement this
  - Given a point in the scene x_1
    - Initially raytrace from the camera
    - Will generalize later



Light Transport

- How to implement this
  - Sample a light source l_i



Light Transport

- Sampling Lights
  - Scenes contain many lights
  - Need to pick which one to sample
  - Multiple options
    - Uniform
    - Weighted

Light Transport

- Sampling Lights: Uniform
  - Already know how to do this!
  - PMF for I'th light: $pmf(l_i) = \frac{1}{N_{lights}}$
  - Sampling: $l_i = \lfloor \xi \cdot N_{lights} \rfloor$
- Implement this in Scene class in Scene.h

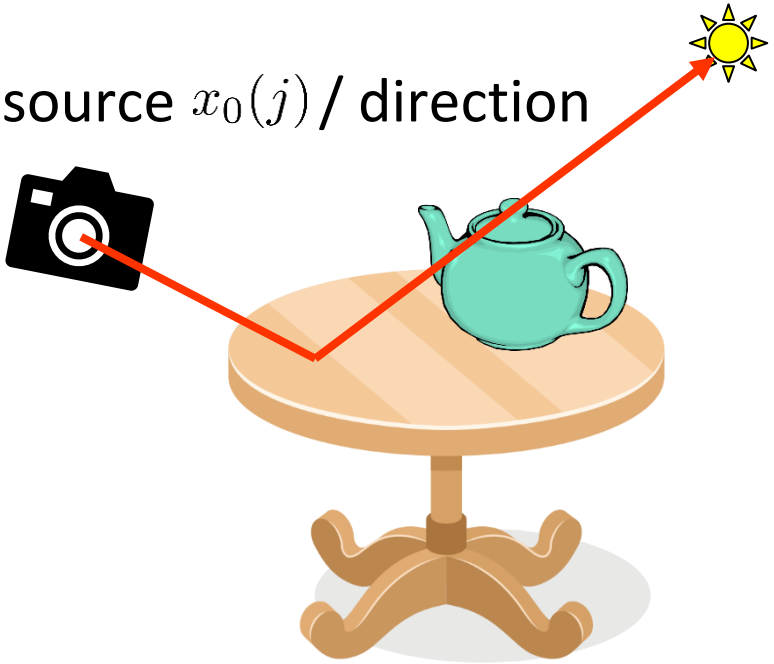
```
Light* sampleLight(Sampler* sampler, float& pmf)
```

Light Transport

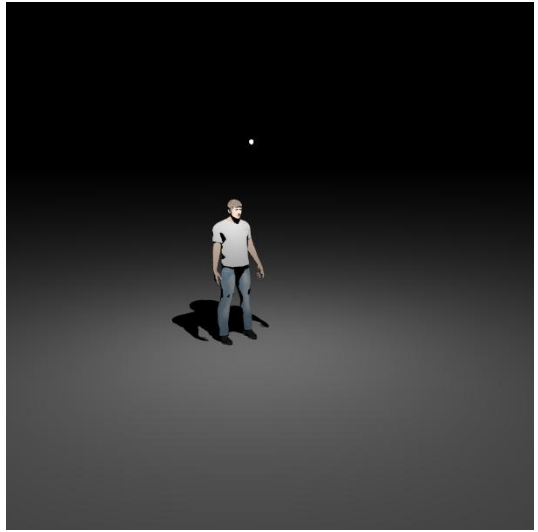
- Sampling Lights: Weighted
  - Can sample proportion to power
  - Variance reduction
  - PMF: $$pmf(l_i) = \frac{\Phi_i}{\sum_{j=1}^{N_{lights}} \Phi_j}$$
  - Need to sample from 1D distribution
  - Can precompute PDF / CDF on startup

Light Transport

- How to implement this
  - Sample a point on the light source $x_0(j)$ / direction to the light source $\omega_i(j)$



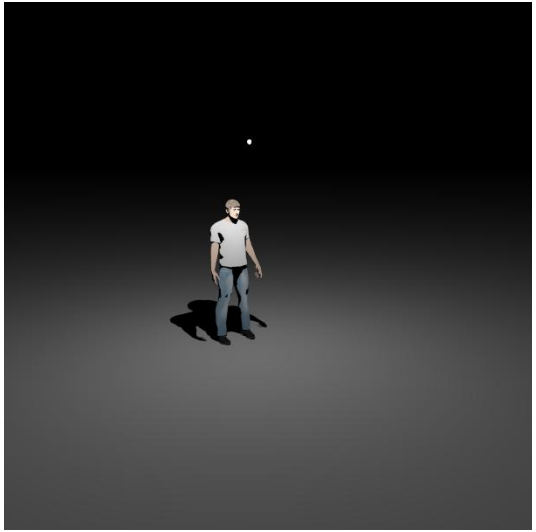
Light Transport



- Point Lights
  - Defined as a point x
  - $\frac{1}{r^2}$ falloff
  - Singularity as $\lim_{r \to 0} \frac{1}{r^2}$
  - Can be uniform emission or follow profile
  - Total emitted light $\Phi = \int_{\mathcal{S}^2} L_e(\mathbf{x}, \omega_o) d\omega_o = 4\pi L_e$

Light Transport

- Point Lights
  - Sample point on light x_l
    - $x_l = x$
  - PDF $p_A(x_l) = 1$



Light Transport

- Directional Lights
  - Defined as a direction ω_l
  - Radiance $L_e\delta(\omega - \omega_l)$

  - Total emitted light proportional to extents of the scene A_{total}: $\Phi = L_e A_{total}$

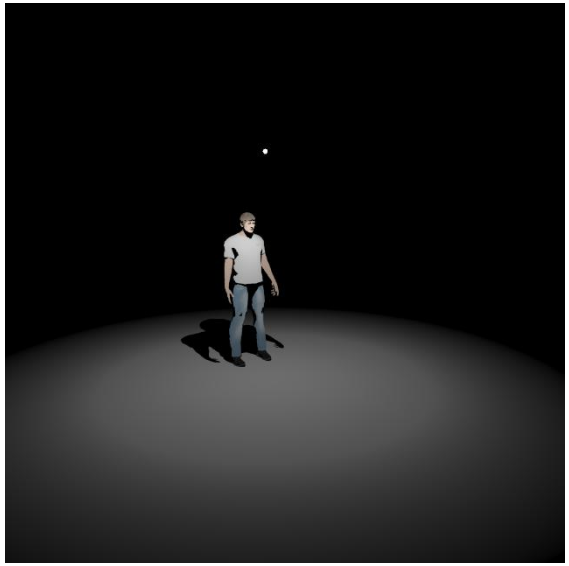
Light Transport

- Point Lights
  - Cannot sample point
  - But can sample direction ω_l

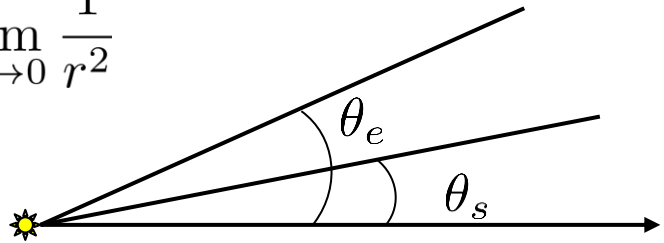
$$\omega = \omega_l$$

  - PDF $p_\Omega(\omega) = 1$

Light Transport

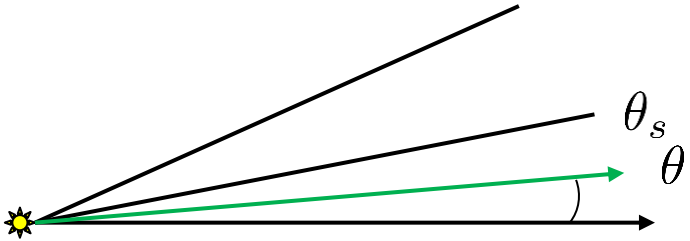


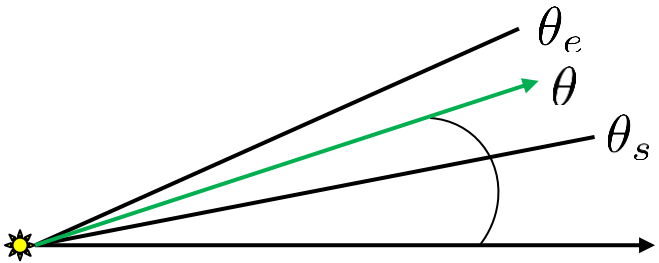
- Spot Lights
  - Defined by a point x , direction and angles $\theta_s \quad \theta_e$
    - Sampling same as point lights
  - $\frac{1}{r^2}$ falloff and singularity as $\lim_{r \to 0} \frac{1}{r^2}$



Light Transport

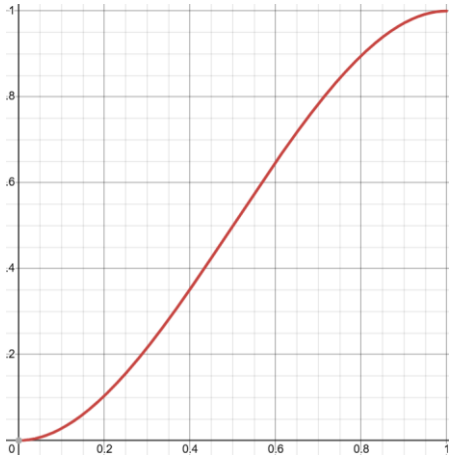
- Spot Lights
  - Full intensity when $\theta < \theta_s$
  - Smoothstep when $\theta_s < \theta < \theta_e$
  - 0 otherwise





Light Transport

- Spot Lights
  - Smoothstep $t^2(3-2t)$
    - Where $t = \frac{\theta - \theta_s}{\theta_e - \theta_s}, t \in [0,1]$



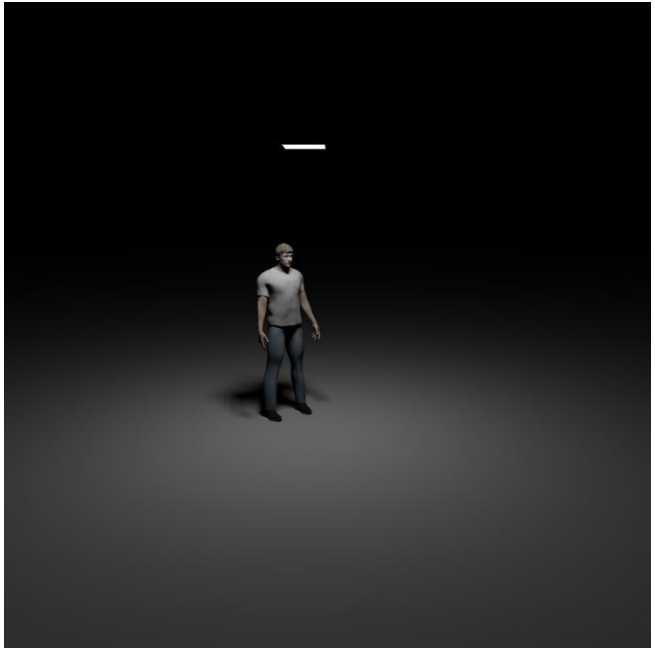
  - Emitted power (assuming uniform emission)

$$\Phi = L_e 2\pi \left[\int_0^{\theta_s} \sin\theta d\theta + \int_{\theta_s}^{\theta_e} smoothstep(\theta, \theta_s, \theta_e) d\theta \right] = L_e 2\pi (1 - \cos(\theta_s)) \frac{\theta_e - \theta_s}{2}$$

Light Transport

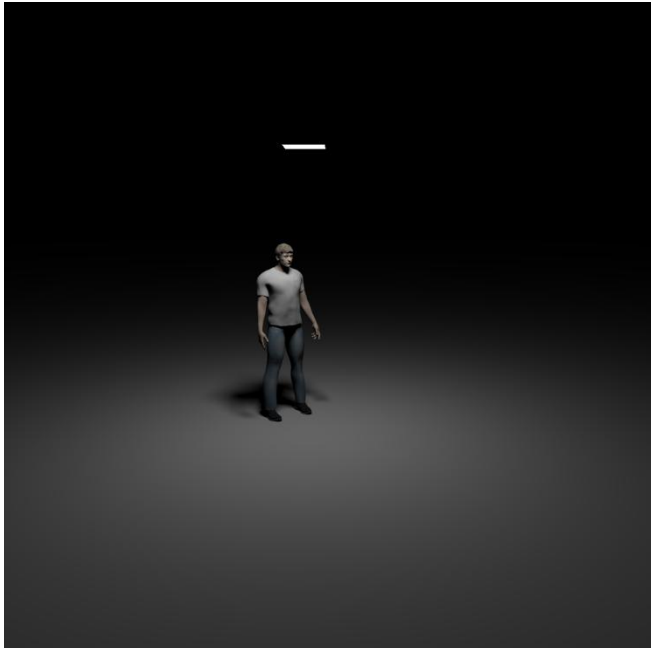
- Area Lights
  - Geometry emits light
  - More realistic
  - Emitted light proportional to surface area

$$\Phi = L_e A$$



Light Transport

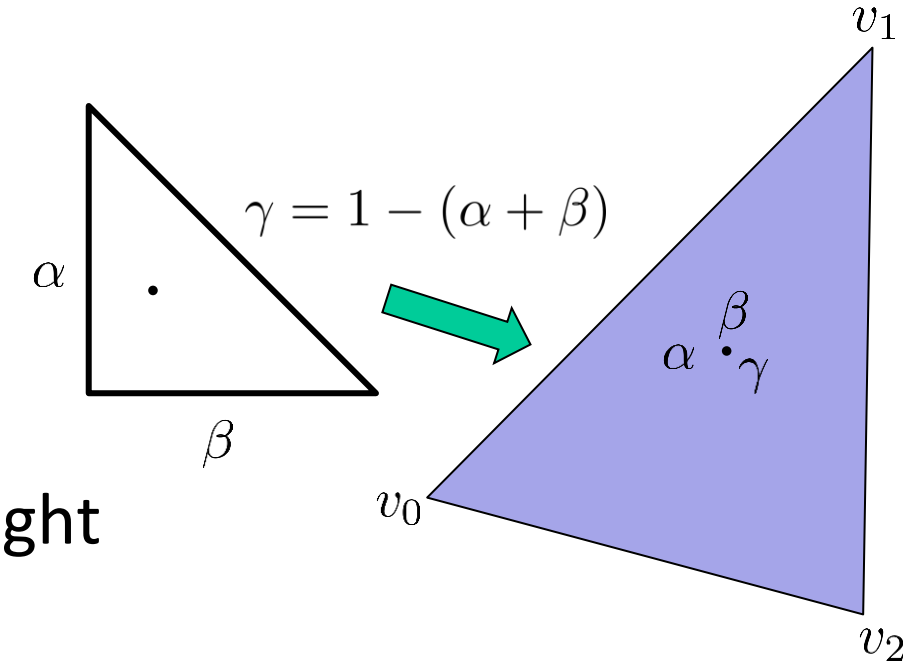
- Area Lights
  - Sampling
  - Uniform pick point on light area
    - We have seen how to uniform sample triangle
  - PDF $p_A(x_l) = \frac{1}{A}$



Light Transport

- Area Lights
  - Sampling
  - Sample barycentric coordinates (α, β, γ)
  - Convert to point on light

$$x_l = \alpha v_0 + \beta v_1 + \gamma v_2$$



Light Transport

- Environment Lighting
  - Infinitely distant lighting

Light Transport

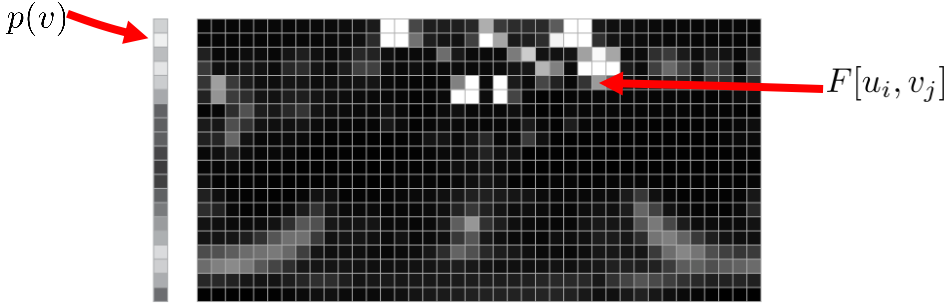
- Environment Lighting
- Usually stored in environment maps
  - Latitude-Longitude

Light Transport

- Environment Lighting
  - Lookup
  - Need map $\omega = (\theta, \phi) \mapsto (u, v)$
    - i.e. $(u, v) = \left(\frac{\phi}{2\pi}, \frac{\theta}{\pi} \right)$
  - So $L_e(\mathbf{x}, \omega_o) = F[u, v] = F\left[\frac{\phi}{2\pi}, \frac{\theta}{\pi} \right]$
  - Implemented in Lights.h

Light Transport

- Environment Lighting
  - Sampling
  - First need PDF
    - Start with tabulated PDF $p(u, v)$



Light Transport

- Environment Lighting
  - Need to convert from (u, v) space to (θ, ϕ)
    - i.e. $(\theta, \phi) = (\pi v, 2\pi u)$
    - Jacobian of transform $\left| \begin{matrix} \frac{\partial \theta}{\partial u} & \frac{\partial \theta}{\partial v} \\ \frac{\partial \phi}{\partial u} & \frac{\partial \phi}{\partial v} \end{matrix} \right| = \left| \begin{matrix} 0 & \pi \\ 2\pi & 0 \end{matrix} \right| = 2\pi^2$
  - So $p(\theta, \phi) = \frac{p(u, v)}{2\pi^2}$
  - And $p_\Omega(\omega) = \frac{p(\theta, \phi)}{\sin(\theta)} = \frac{p(u, v)}{2\pi^2 \sin(\theta)}$

Light Transport

- Environment Lighting
  - Generates samples uniformly in (u, v) space
  - Wastes samples at poles
  - Can incorporate $\sin(\theta)$ into $F[u, v]$
    - Do this when creating tabulated distribution

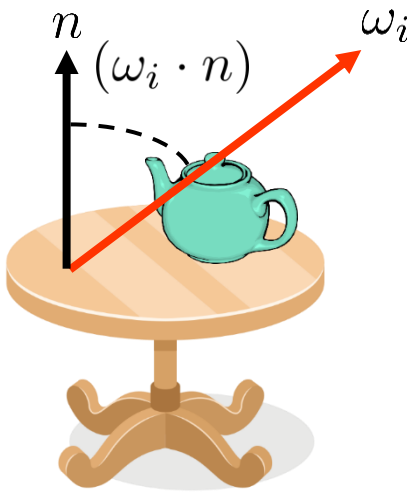
$$F'[u, v] = F[u, v] \sin(\theta) = F[u, v] \sin(\pi v)$$

  - Approximately cancels $\sin(\theta)$ in denominator!

Light Transport

- Environment Lighting
  - Sampling
    - Sample (u, v) from tabulated distribution
    - Convert to spherical coordinates $(\theta, \phi) = (\pi v, 2\pi u)$
    - Convert to direction $(\cos(\phi)\sin(\theta), \cos(\theta), \sin(\phi)\sin(\theta))$
      - Note y-up
  - PDF $p_\Omega(\omega) = \frac{p(u,v)}{2\pi^2 \sin(\theta)}$

Light Transport



- Environment Lighting
  - Geometry term
  - Not well defined
    - Special case
    - Only use direction from surface and visibility

$$G(x_i \leftrightarrow x_{i+1}) = (\omega_i \cdot n)V(x_i \leftrightarrow x_{i+1})$$

Light Transport

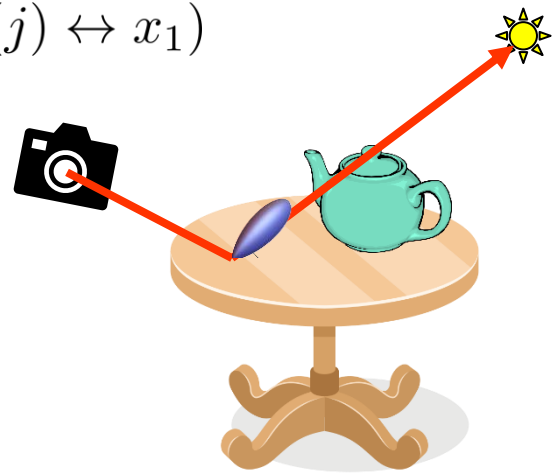
- Environment Lighting
  - Total Emitted Light

$$\Phi = A \int_{\mathcal{S}^2} L_e(\mathbf{x}, \omega_o) d\omega_o$$

Light Transport

- How to implement this
  - Evaluate integrand

$$f_r(x_0(j) \rightarrow x_1 \rightarrow x_2)L(x_0(j) \rightarrow x_1)G(x_0(j) \leftrightarrow x_1)$$



Light Transport

- How to implement this
  - Evaluate integrand
  - Evaluate BSDF $f_r(x_0(j) \rightarrow x_1 \rightarrow x_2)$
  - Evaluate Light $L(x_0(j) \rightarrow x_1)$
  - Evaluate Geometry Term $G(x_0(j) \leftrightarrow x_1)$

Light Transport

- How to implement this
  - Evaluate Geometry Term $G(x_0(j) \leftrightarrow x_1)$
    - If area light $G(x_i \leftrightarrow x_{i+1}) = \frac{\cos(\theta)\cos(\theta')}{r^2} V(x_i \leftrightarrow x_{i+1})$
    - If directional / environment light

$$G(x_0 \leftrightarrow x_1) = \cos(\theta) V(x_0 \leftrightarrow x_1)$$

      - Because no “next” surface
      - Visibility to point outside scene in direction $x_0 \rightarrow x_1$
      - Use the scene bounding box

Light Transport

- How to implement this
  - One important exception: Some surfaces cannot directly connect to the light
    - Specular surfaces
  - Only non-specular surfaces can connect
    - Must check `shadingData.bsdf->isPureSpecular() == false`

Monte Carlo Rendering

- Direct Lighting Implementation
- Add in Renderer class

```
Colour computeDirect(ShadingData shadingData, Sampler* sampler)
```

- Call from `Colour direct(Ray& r, Sampler* sampler)`
  - Need to add raytracing from camera code and call `computeDirect`

Monte Carlo Rendering

- Useful files:
  - Materials.h contains BSDFs
  - Sampling.h contains sampling code
  - Lights.h contains Lights

Monte Carlo Rendering

- Direct Lighting Implementation
  - First sample light from

```
Light* sampleLight(Sampler* sampler, float& pmf)
```

  - Check if light is area or environment map

```
if (light->isArea())
```

  - Light sampling definition

```
Vec3 sample(const ShadingData& shadingData, Sampler* sampler, Colour& emittedColour, float& pdf)
```

Monte Carlo Rendering

- Direct Lighting Implementation
  - If area:
    - Sample point on light and store returned emission
    - Calculate Geometry Term
    - Calculate visibility
    - Evaluate BSDF `shadingData.bsdf->evaluate(shadingData, wi)`
    - Multiply terms and return

Monte Carlo Rendering

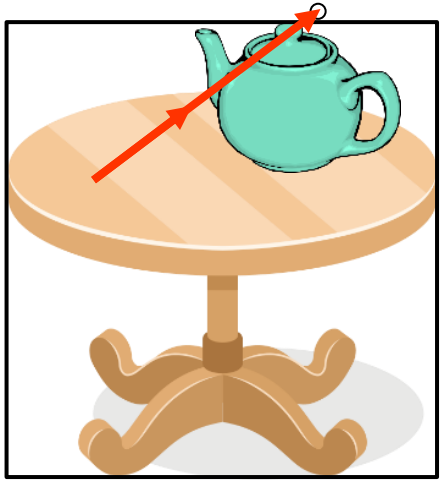
- Direct Lighting Implementation
  - If environment map:
    - Sample from light, returns direction instead of point

```
Vec3 sample(const ShadingData& shadingData, Sampler* sampler, Colour& emittedColour, float& pdf)
```

    - Evaluate Geometry Term for environment maps
    - Evaluate visibility to outside scene bounds
    - Evaluate BSDF and multiply terms and return

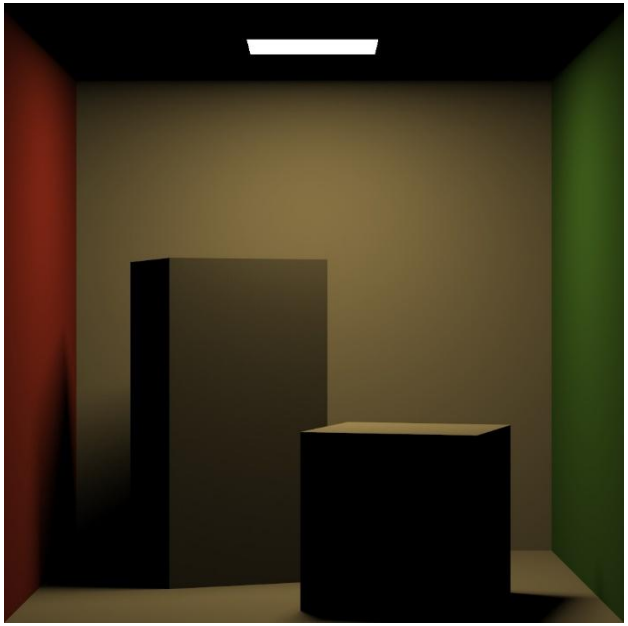
Monte Carlo Rendering

- Direct Lighting Implementation
  - If environment map:
    - Visibility



Monte Carlo Rendering

- Direct Lighting Implementation

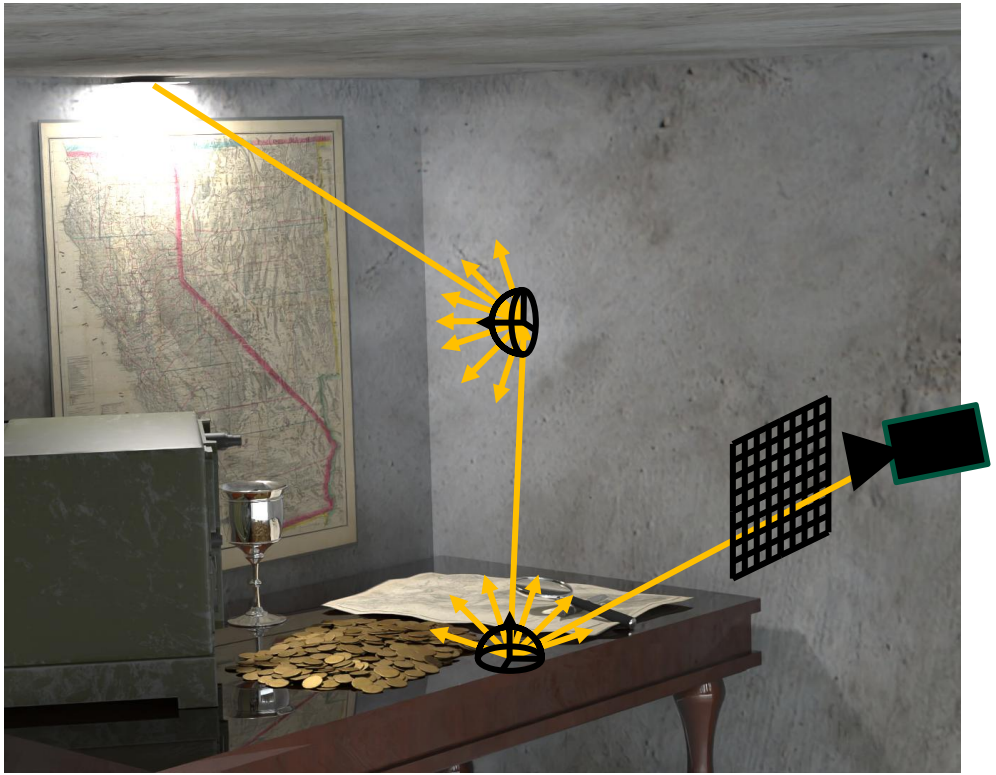


Light Transport

- Indirect lighting
  - Recall $L_o = L_e + L_d + L_{ind}$
  - Where does L_{ind} come from?
  - Light Paths
  - Our equations do not express this concept yet
  - Need a new formulation

Light Transport

- Light Paths



Light Transport

- Path Integral Formulation
  - Integral over paths $I = \int_{\mathcal{P}} f(\bar{x}) d\mu(\bar{x})$
  - $\mathcal{P}$ is path space
  - $\bar{x}$ is a light path
  - $d\mu(\bar{x})$ is product area measure

Light Transport

- Path Integral Formulation $I = \int_{\mathcal{P}} f(\bar{x}) d\mu(\bar{x})$
  - Consider space of all paths of a fixed length $\mathcal{P}_L$
    - Each space is a cartesian outer product of scene manifolds $\mathcal{P}_L = \bigtimes_{i=0}^{L} \mathcal{M}_i = \mathcal{M}_0 \times \mathcal{M}_1 \times ... \times \mathcal{M}_L$
  - Domain is union of each length

$$\mathcal{P} = \bigcup_{i=2}^{\infty} \mathcal{P}_i$$

Light Transport

- Path Integral Formulation $I = \int_{\mathcal{P}} f(\bar{x}) d\mu(\bar{x})$
  - Consider space of all paths of a fixed length $\mathcal{P}_L$
  - Product Area Measure (on surfaces)

$$\mu_L(D \subseteq \mathcal{P}_L) = \int_D \prod_{i=0}^{L} dA(x_i) \text{ so } \; d\mu_L(\bar{x}) = dA(x_0)dA(x_1)...dA(x_L)$$

  - Combine

$$I = \sum_{i=1}^{\infty} \int_{\mathcal{P}_i} f(\bar{x}) d\mu_i(\bar{x}) = \int_{\mathcal{P}} f(\bar{x}) d\mu(\bar{x}) \qquad \mu(D) = \sum_{i=1}^{\infty} \mu_i(D \cap \mathcal{P}_i)$$

Light Transport

- Path Integral Formulation $I = \int_{\mathcal{P}} f(\bar{x}) d\mu(\bar{x})$
  - Path defined as $\bar{x} = x_0 x_1 ... x_L$
  - Path contribution defined as product of interactions between light and camera

$$f(\bar{x}) = L_e(x_0 \rightarrow x_1) \left[\prod_{i=1}^{L-2} G(x_{i-1} \leftrightarrow x_i) f_r(x_{i-1} \rightarrow x_i \rightarrow x_{i+1}) \right] G(x_{L-1} \leftrightarrow x_L) W_e(x_{L-1} \rightarrow x_L)$$

Light Transport

- Monte Carlo Estimate (Path Tracing)
  - Estimate of a single path of length L

Sample next path vertex

$$\frac{L_e(x_0 \to x_1)}{p_A(x_0)} \left[\prod_{i=1}^{L-2} \frac{G(x_{i-1} \leftrightarrow x_i) f_r(x_{i-1} \leftarrow x_i \leftarrow x_{i+1})}{p_A(x_{i-1} \leftarrow x_i)} \right] \frac{G(x_{L-1} \leftrightarrow x_L) W_e(x_{L-1} \leftarrow x_L)}{p_A(x_{L-1} \leftarrow x_L)}$$

Sample Light

Sample scene from camera

Light Transport

- Monte Carlo Estimate (Path Tracing)
- Note, connection between PDF w.r.t solid angle $p_\Omega(x_{i-1} \leftarrow x_i)$ and area $p_A(x_{i-1} \leftarrow x_i)$
  - Projected solid angle PDF $p^\perp(x_{i-1} \leftarrow x_i) = \frac{p_\Omega(x_{i-1} \leftarrow x_i)}{\cos(\theta)}$
  - PDF w.r.t. area: $p_A(x_{i-1} \leftarrow x_i) = p^\perp(x_{i-1} \leftarrow x_i) G(x_{i-1} \leftrightarrow x_i)$

Light Transport

- Monte Carlo Estimate (Path Tracing)
  - Estimate of a single path of length L
  - Can rewrite

$$\frac{L_e(x_0 \leftarrow x_1)}{p_A(x_0)} \left[\prod_{i=1}^{L-2} \frac{G(x_{i-1} \leftrightarrow x_i) f_r(x_{i-1} \leftarrow x_i \leftarrow x_{i+1})}{G(x_{i-1} \leftrightarrow x_i) p^{\perp}(x_{i-1} \leftarrow x_i)} \right] \frac{G(x_{L-1} \leftrightarrow x_L) W_e(x_{L-1} \leftarrow x_L)}{G(x_{L-1} \leftrightarrow x_L) p^{\perp}(x_{L-1} \leftarrow x_L)}$$

Light Transport

- Monte Carlo Estimate (Path Tracing)
  - Sample Next Vertex
  - Local sampling decision $\frac{G(x_{i-1} \leftrightarrow x_i) f_r(x_{i-1} \leftarrow x_i \leftarrow x_{i+1})}{G(x_{i-1} \leftrightarrow x_i) p^{\perp}(x_{i-1} \leftarrow x_i)}$
  - Simplifies $\frac{f_r(x_{i-1} \leftarrow x_i \leftarrow x_{i+1}) \cos(\theta)}{p_\Omega(x_{i-1} \leftarrow x_i)} = \frac{f_r(x, \omega_i, \omega_o) \cos(\theta)}{p_\Omega(\omega_i)}$
    - Can use solid angle sampling
    - Geometry term cancels

Light Transport

- Monte Carlo Estimate (Path Tracing)
  - Path Throughput $C(\bar{x}_L)$

$$C(\bar{x}_L) = \prod_{i=1}^{L-2} \frac{G(x_{i-1} \leftrightarrow x_i) f_r(x_{i-1} \leftarrow x_i \leftarrow x_{i+1})}{G(x_{i-1} \leftrightarrow x_i) p^{\perp}(x_{i-1} \leftarrow x_i)}$$

  - Or with the previous formulation

$$C(\bar{x}_L) = \prod_{j=1}^{L-2} \frac{f_r(x_j, \omega_{i,j}, \omega_o) \cos(\theta_j)}{p_\Omega(\omega_{i,j})}$$

Use this in practice

Light Transport

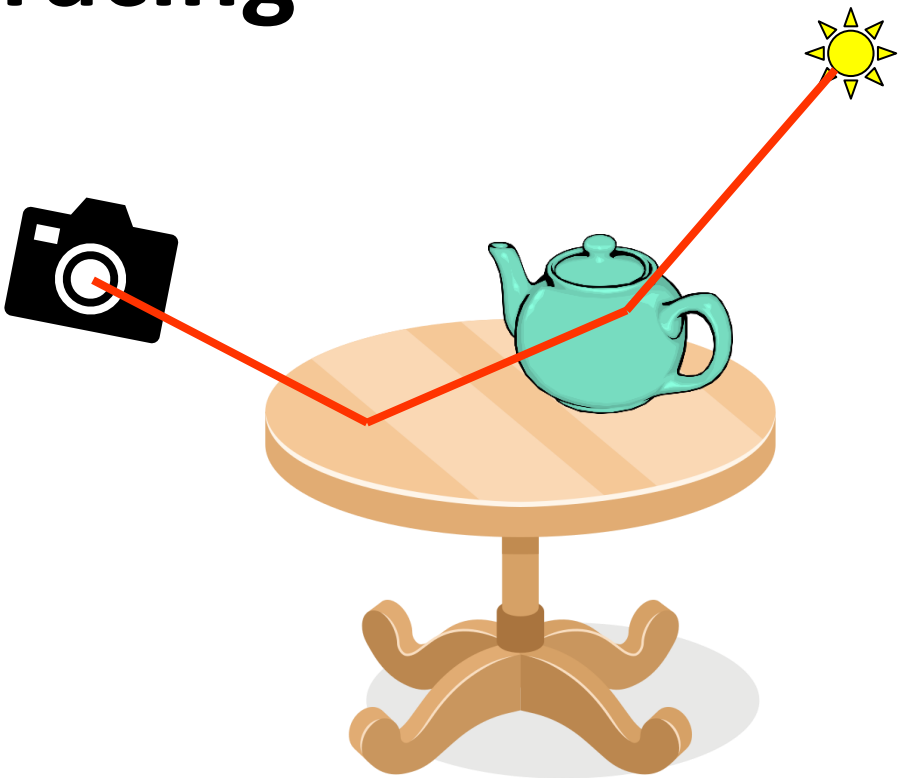
- Monte Carlo Estimate (Path Tracing)
  - Path Throughput

$$C(\bar{x}_L) = \prod_{i=1}^{L-2} \frac{G(x_{i-1} \leftrightarrow x_i) f_r(x_{i-1} \leftarrow x_i \leftarrow x_{i+1})}{G(x_{i-1} \leftrightarrow x_i) p^{\perp}(x_{i-1} \leftarrow x_i)}$$

  - Putting it together

$$I \approx \frac{L_e(x_0 \leftarrow x_1)}{p_A(x_0)} C(\bar{x}_L)$$

Path Tracing



Light Transport

- How to select L?
  - Pick a value when creating path?
    - Tricky to estimate a good value
  - Stop at a certain point?
    - Only samples a subset of path space
  - Stochastically
    - Computes more samples in brighter regions

Light Transport

- How to select L?
  - Stochastically
  - Russian Roulette
    - Stop tracing with probability q and use a value c instead

$$I' = \begin{cases} \frac{I - qc}{1 - q} & if \xi < q \\ c & otherwise \end{cases}$$

    - Usually $c = 0$

Light Transport

- Russian Roulette
  - Proof estimator is unbiased

$$E[I'] = (1-q)\frac{E[I]-qc}{1-q} + qc = E[I]$$

  - Values:
    - Want q to be high in areas of high contribution, low in areas of small contribution $q = min(C^*(\bar{x}_i), 1.0)$
      - Where $C(\bar{x}_i)$ is the path throughput so far
      - Use luminance as scalar is needed $C^*(\bar{x}_i)$

Light Transport

- Path Tracing with Russian Roulette

Russian Roulette

Sample next path vertex

$$\frac{L_e(x_0 \to x_1)}{p_A(x_0)} \left[\prod_{i=1}^{L-2} \frac{1}{1-q_i} \frac{G(x_{i-1} \leftrightarrow x_i) f_r(x_{i-1} \leftarrow x_i \leftarrow x_{i+1})}{p_A(x_{i-1} \leftarrow x_i)} \right] \frac{G(x_{L-1} \leftrightarrow x_L) W_e(x_{L-1} \leftarrow x_L)}{p_A(x_{L-1} \leftarrow x_L)}$$

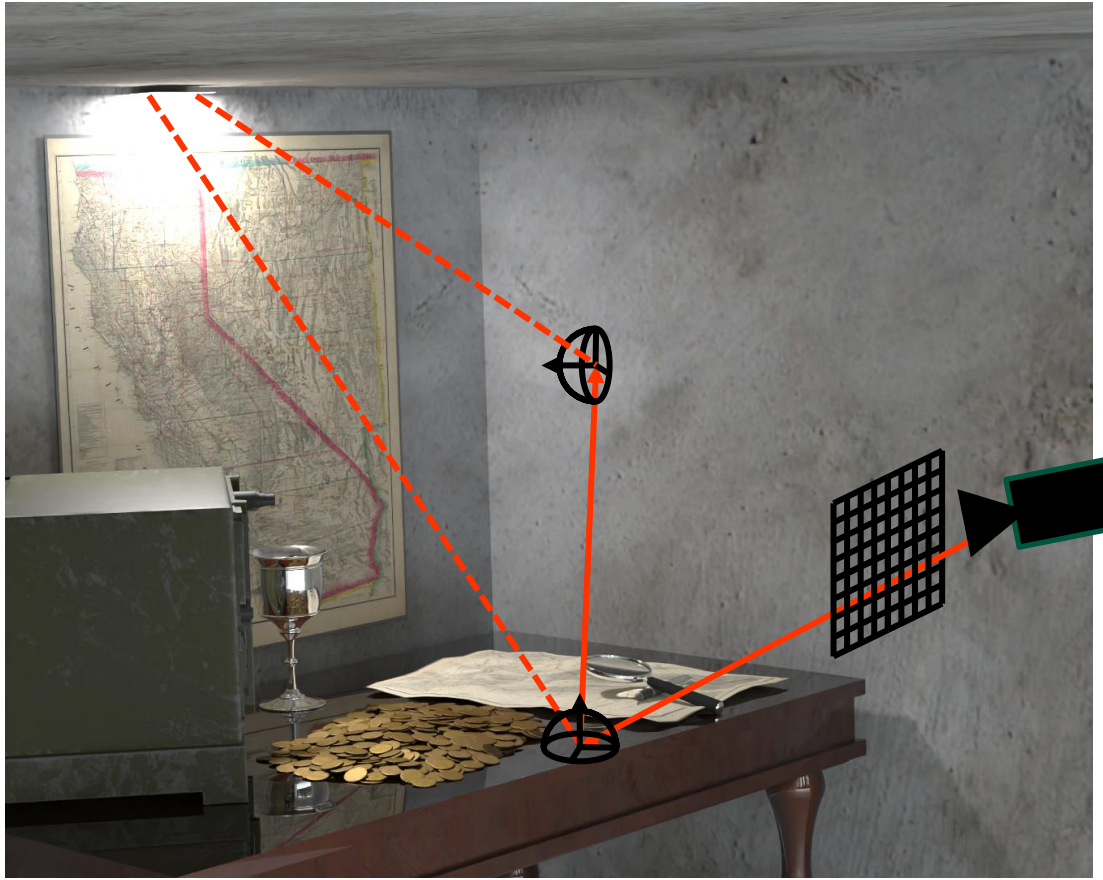
Sample Light

Sample scene from camera

Path Tracing

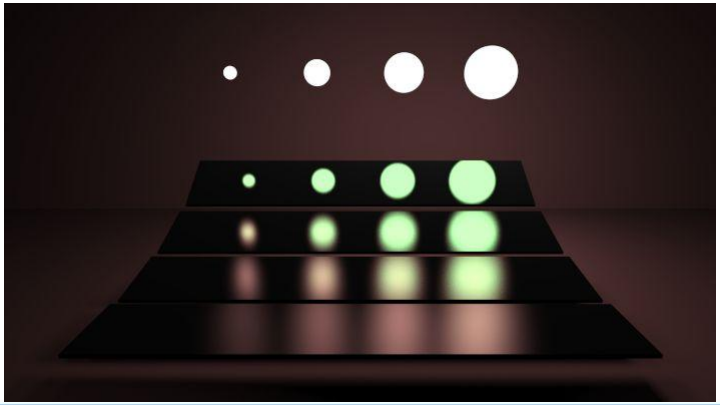
- Can improve efficiency by progressively building paths with shared prefix
- At each interaction, compute direct, and stochastically extend path

Path Tracing



Path Tracing

- What happens if an indirect sample hits a light?
- Have to be careful not to double count



Path Tracing

- Option 1:
  - Only contribute when
    - L=1 (i.e. the camera ray directly hits the light)
    - Or when the light is viewed in a specular surface

```
shadingData.bsdf->isPureSpecular() == true
```

    - Have to track the previous surface type

Path Tracing

- Option 2:
  - Combine with Multiple Importance Sampling (MIS)
    - Apply MIS when calculating direct and indirect lighting

Path Tracing

- MIS when computing direct

$$L_d(x_1 \to x_{n+1}) =$$

$$\frac{1}{N}\sum_{j=1}^{N} w_d \frac{f_r(x_0(j) \to x_1 \to x_2) L(x_0(j) \to x_1) G(x_0(j) \leftrightarrow x_1)}{p_A(x_0(j))}$$

  - Where $w_d = \dfrac{p_A(x_0)}{p_A(x_0) + p_A(x_1 \to x_0)}$
  - And as domains must be the same

$$p_A(x_1 \to x_0) = \frac{p_\Omega(x_1 \to x_0)}{\cos(\theta)} G(x_1 \leftrightarrow x_0)$$

Path Tracing

- MIS when computing indirect and hits light

$$L_{ind}(x_1 \to x_{n+1}) = \frac{1}{N}\sum_{j=1}^{N} w_{ind}\frac{f_r(x,\omega_{i,j},\omega_o)\cos(\theta_j)}{p_\Omega(\omega_{i,j})}$$

  - Where $w_{ind} = \frac{p_A(x_1 \to x_0)}{p_A(x_0) + p_A(x_1 \to x_0)}$
  - Note if a specular surface $p_\Omega(\omega_{i,j}) = 1$ only if specular surface sampled, so same as before

Monte Carlo Rendering

- Path Tracing
  - For each pixel
    - Sample a point in the pixel
    - 1. Trace ray
    - Calculate direct lighting
    - Sample indirect direction, update throughput
    - Goto 1. until path terminated

Monte Carlo Rendering

- Indirect Lighting
  - For each pixel
    - Sample a point in the pixel

```
float px = x + sampler->next();
float py = y + sampler->next();
Ray ray = scene->camera.generateRay(px, py);
```

    - Starting point of path
- Recall integrating over pixel area

Monte Carlo Rendering

- Path Tracing
  - Sample indirect direction, update throughput

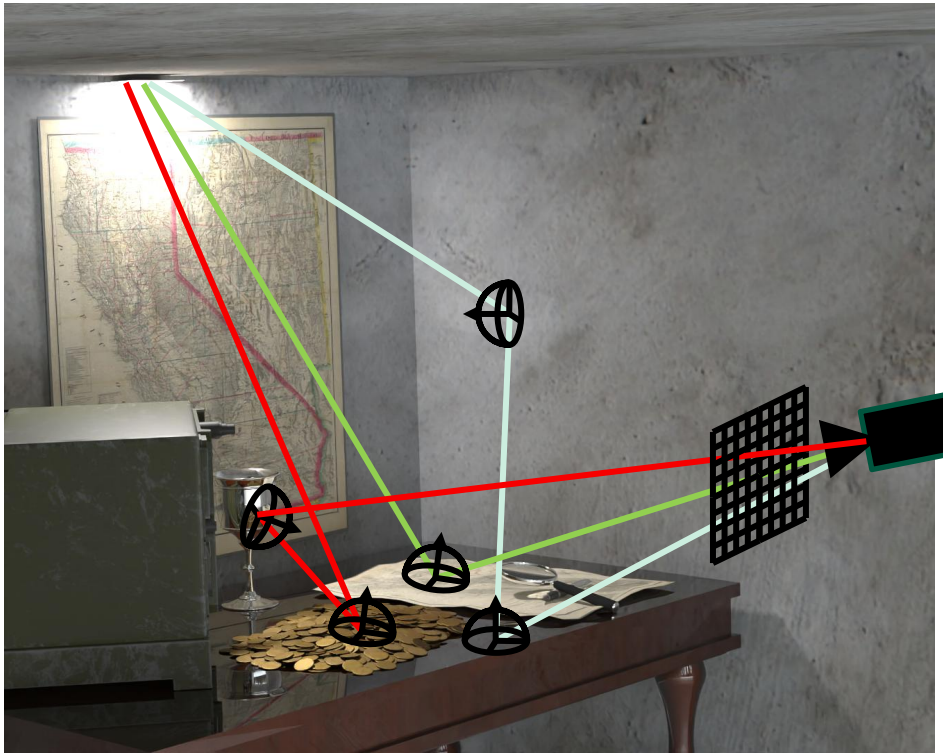
  – Any PDF over the hemisphere can be used $p(\omega_i)$

  – Update path throughput

$$C(\bar{x}_{i+1}) = C(\bar{x}_i) \frac{f_r(x_i, \omega_o, \omega_i) \cos(\theta)}{p(\omega_i)}$$

Monte Carlo Rendering

- Path Tracing
  - Average over many paths



Monte Carlo Rendering

- Path Tracing Implementation
  - In Renderer.h

```
Colour pathTrace(Ray& r, Colour& pathThroughput, int depth, Sampler* sampler)
```

$C(\bar{x}_L)$

Initialize with 0, increment each bounce

Sampler per thread

  - Can change definition if needed

Monte Carlo Rendering

- Path Tracing Implementation
  - Initialize `Colour pathThroughput(1.0f, 1.0f, 1.0f);`
  - Include direct (NEE) from

    ```
    Colour computeDirect(ShadingData shadingData, Sampler* sampler)
    ```

  - Add the recursively computed indirect lighting to this and return

Monte Carlo Rendering

- Path Tracing Implementation
  - Sample incoming radiance from

```
static Vec3 uniformSampleHemisphere(float r1, float r2)
```

    - From `class SamplingDistributions` in Sampling.h
  - Samples z-up
  - Convert to world via `wi = shadingData.frame.toWorld(wi);`

Monte Carlo Rendering

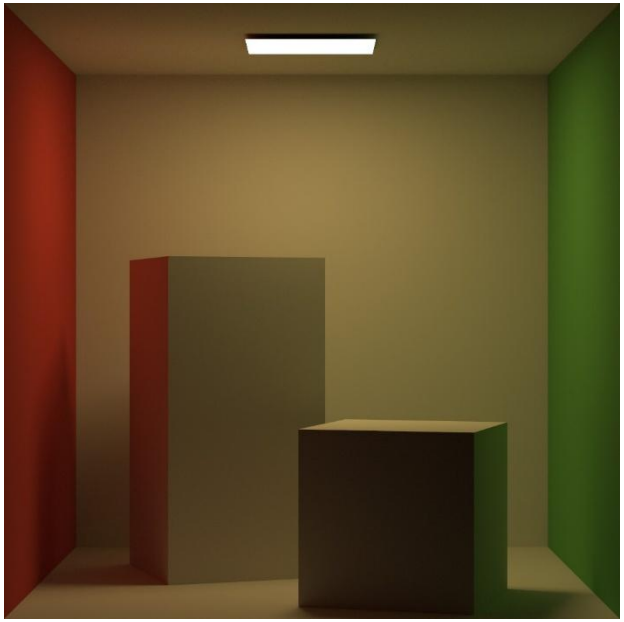
- Path Tracing Implementation
  - Multiply with BSDF and cosine, divide by PDF

```
static float uniformHemispherePDF(const Vec3 wi)
```

    - From Sampling.h
  - Also need to calculate Russian Roulette
  - Recursively call `pathTrace` to compute indirect

Monte Carlo Rendering

- Path Tracing Implementation



Questions?

Computer Graphics

Light Transport and Path Tracing

Dr. Tom Bashford-Rogers

WARWICK